

Multi-resolution semantic abstraction over the TKH hypergraph

Ilia Dzeladze, 2026-09-21

I designed and directed this project end to end, using a coding agent (Claude Code) to implement, run, and debug the pipeline under my direction. For a few parts past what I'd expect to know starting this, mainly the hyperedge weighting and the dendrogram monotonicity argument, and a couple of subtle correctness bugs, I worked through the reasoning with the agent directly rather than just accepting output. Specifics on tool use and prompts are in `AI_USAGE.md`.

0. Formal statement

Objective. Given a hypergraph snapshot $H(t) = (V(t), E(t))$ restricted to "concept" node types (method, technique, task, problem, dataset, metric, component, cited_work, future_topic, claim; article and author shape structure but never become super-node members themselves), build a laminar sequence of partitions $P_0, \dots, P_K$ of $V(t)$ by agglomerative clustering on a single joint affinity graph

$$A(i, j) = \alpha \cdot S(i, j) + (1 - \alpha) \cdot M(i, j)$$

where S is a structural affinity from a weighted clique expansion of the hypergraph (each hyperedge e of arity n contributes $1/(n-1)$ to every pair of its endpoints, so each member gets total weight 1 per hyperedge; this is a pairwise projection, close to but not the same as the clique expansion implied by Zhou, Huang and Schoelkopf's normalized hypergraph Laplacian) and M is a semantic k -NN affinity over sentence embeddings of node surface_form text. Both are normalized to $[0,1]$ independently before combining, $\alpha = 0.3$ in the final run, chosen by ablation (section 3) rather than left at the 0.5 I started with. The dendrogram from average-linkage clustering on A is cut at three target sizes (12, 50, 200) to produce three super-node levels; the tree itself is the laminar hierarchy, cuts of one dendrogram at different heights are always nested in each other.

The structure/semantics trade-off (P3). I resolved this as a single joint objective rather than a two-stage or consensus scheme, mainly because it collapses the trade-off into one literal, sweepable parameter (α) instead of a qualitative staging decision about which signal gets to act first. I considered a two-stage alternative (spectral structural clustering fixes the coarse level, semantics only refines within it) and rejected it: it's computationally convenient, but it means the coarse level is determined by structure alone, so a case where two topically identical but citation-disconnected clusters should merge at a coarse level has no path to do so. The joint graph lets a strong semantic signal override weak structural evidence at any level, not just at the bottom. The real cost of the joint approach, as originally shipped, was that α itself was never properly ablated, it sat at 0.5 by default. I closed that specific gap afterward with a full sweep (section 3), which is how the final run ended up at $\alpha=0.3$ instead. The two-stage alternative above is still only reasoned about, not actually implemented and measured against the shipped method; that comparison remains a real gap (section 5).

Complexity. Structural affinity construction is $O(\sum_e |e|^2)$ in the worst case (bounded by roughly 66k nonzero pairs on the largest snapshot, 5428 concept nodes, 1429 hyperedges). The semantic k -NN graph is a $k=15$ nearest-neighbor search over 768-dim embeddings, dominated in practice by the sentence-embedding forward pass itself (about 107 of 110 seconds on the full 2026 snapshot) rather than the neighbor search. The sparse UPGMA merge is $O(E \log E)$ via a heap with lazy deletion, and empirically fast, about 1.6 seconds for 5427 merges on the same snapshot. Wall-clock at this corpus size is embedding-bound, not algorithm-bound.

Guaranteed by construction vs. measured empirically.

- P1 (laminar refinement): by construction. A dendrogram's cuts are nested by definition; `validate_hierarchy.py` checks this exactly (every super-node's children partition its members with zero gap or overlap) across all four snapshots, not just assumed from the algorithm's shape.
- P2 (size budget, N_0 in $[10,15]$): by construction. `cut_to_k_clusters` always returns at most the target count; $N_0 = 12$ at every snapshot.
- P3 (semantic coherence): empirical, not guaranteed. The joint-affinity weighting is a design choice with no mathematical guarantee that meaning wins out over noise. What's measured (section 4 below) is strongly positive, but that's a result, not a property of the construction.
- P4 (hyperedge fidelity): only partly met. The clustering runs on a weighted clique expansion, which is a pairwise projection, with arity-aware weights so big hyperedges don't dominate. The T4 collapse rule keeps a k -ary relation as one coarse hyperedge when it spans three or more super-nodes, and it's implemented and

unit tested. But the collapsed hypergraph is only written to the output in the shipped version. A variant where it drives the coarser levels was built and tested but lost on coherence (section 2).

- P5 (temporal stability): empirical, and deliberately not optimized for. The matching mechanism (Jaccard overlap across independently-clustered snapshots) only measures stability after the fact, it does nothing during clustering to encourage it. Measured cross-snapshot ARI ranges 0.35 to 0.65 depending on level (section 4).
- P6 (interpretable, faithful, temporally honest labels): mixed. Temporal honesty for the labeller's INPUT is by construction, a label for year t structurally cannot see member text the corpus hadn't ingested by t , because that node was never included in the snapshot in the first place. Whether the label itself avoids overclaiming given that input is empirical and imperfect. Checked against members the labeller never saw, 5-14% of glosses are contradicted and 60-79% are not entailed, depending on the snapshot (section 3).

1. Method and where it sits in the literature

The structural affinity is a weighted clique expansion. Zhou, Huang and Schoelkopf (2006) define hypergraph spectral clustering through a normalized hypergraph Laplacian, and Agarwal, Branson and Belongie (2006) showed that Laplacians like theirs reduce to clique or star expansions, so arity-aware weighting of a projection is the part I borrow. I don't run a spectral step. I checked what a naive unweighted clique expansion would have done instead: on the 2026 snapshot, hyperedges of arity greater than 10 hold about 94% of total pairwise weight under the naive version, versus about 63% under the native $1/(n-1)$ weighting. Since roughly 80% of the hyperedges in this corpus have arity greater than 2, that's not a theoretical nicety, a handful of large tables would otherwise dominate the whole structural signal.

I considered hypergraph modularity (Kumar et al. 2020) as an alternative clustering objective, closer in spirit to Louvain-style greedy community detection than to spectral cuts. I didn't use it, mainly because a modularity-maximizing method doesn't naturally hand you a full laminar hierarchy the way a dendrogram does, you'd need to run it recursively at multiple resolutions and then separately verify the results nest properly, which is exactly the kind of "assumed, not verified" risk I was trying to avoid by using average-linkage clustering directly.

The temporal matching mechanism (birth, death, stable, grow, shrink, merge, split) follows the event vocabulary from Greene, Doyle and Cunningham (2010) on tracking community evolution in dynamic networks, applied here via Jaccard overlap between independently-clustered snapshots rather than their original setting. I chose matching over warm-starting the clustering itself (which would be closer to a regularized/temporally-smoothed method) because it keeps every snapshot fully reproducible in isolation. The real cost, stated plainly: matching only measures stability, it never pushes the clustering toward being stable, so whatever temporal coherence shows up in the results is earned by the corpus's own structure and the labels' persistence, not engineered in.

For the coarsening operation itself (T4, next section), the closest published framing is graph reduction with spectral guarantees (Loukas 2019), though that work is about pairwise graphs and doesn't address what happens to an edge that spans three or more coarse groups at once, which is the hypergraph-specific part of T4.

I stuck to citations I could verify the details of rather than reaching for very recent (2023-2026) hypergraph clustering papers I wasn't fully sure I remembered correctly. That's a real limitation of the literature grounding here, not a claim that nothing newer is relevant.

2. The T4 coarsening decision

When a hyperedge's endpoints land across super-nodes after clustering, three cases: all endpoints in one super-node means the edge is now fully internal, dropped from the coarse hypergraph and counted instead as an internal-density statistic; endpoints spanning exactly two super-nodes becomes an ordinary pairwise edge; endpoints spanning three or more super-nodes stays a genuine k -ary hyperedge between those groups. That last case is the one worth defending: the lossy alternative is exploding it into every pairwise combination between the spanned groups, which inflates apparent density by $O(k^2)$ and destroys the fact that all k groups were asserted together by one source fact, not by $k(k-1)/2$ independent facts.

The pipeline applies this rule at every level of every snapshot and writes the result into `hierarchy.json`, but in the shipped version nothing downstream reads it, since all levels are cuts of one dendrogram. I built and tested a multilevel variant where each coarser level is clustered on the T4-collapsed hypergraph of the level below. It lowered level-0 coherence by about a third and put 39-42% of nodes into one level-0 cluster, so under the rule I'd written down beforehand it didn't ship. It did raise level-0 cross-snapshot ARI from 0.35 to 0.43-0.51. Articles and

authors pass through as singleton super-nodes, so edges touching an article can never become internal. On the 2026 snapshot's level-0 hierarchy (12 super-nodes), of 1429 original hyperedges, 592 became coarse edges: 128 pairwise and 464 genuine multi-way hyperedges, only 7 became fully internal and were dropped. What the rule loses: once several original hyperedges collapse onto the same super-node set, they merge into one coarse edge with a weight count and a relation-type breakdown, not a record of which specific original member participated in which specific original edge.

These exact counts shifted after the alpha ablation in section 3 (alpha=0.5 gave 532 coarse edges with 116 internal-dropped at level 0; alpha=0.3 gives 592 with only 7 dropped). Lower alpha weights semantic similarity more and hyperedge co-occurrence less, so nodes that happen to co-occur in the same edge are less likely to all land in one super-node together, which is why far fewer edges end up fully internal. I noticed this as a side effect of rerunning T4 after the ablation, not something I went looking for.

3. Evaluation

Alpha ablation. Section 0 flagged alpha=0.5 as an unswept default. Before trusting any of the numbers below, I went back and swept it: 0.05 through 0.95 in steps of 0.05, 19 values, scored by coherence-vs-null and both stability measures, the two of these four metrics that don't need hand-written labels, so the sweep itself didn't need to relabel anything 19 times over. Averaged across all three levels and all four snapshots, mean coherence z-score rose from 517 under alpha=0.5 to 574 under alpha=0.3, and mean stability (perturbation and cross-snapshot averaged together) rose from 0.54 to 0.62. An aggregate like that isn't a strict enough bar on its own, it can hide a level that got worse while others improved, so I checked the stricter version too: separately at each of the three levels, on each of coherence z-score, cross-snapshot ARI, and perturbation ARI, nine numbers per alpha value. alpha=0.3 was the only value in the entire grid that beat the shipped 0.5 on all nine, with no exceptions. I moved the default to 0.3, regenerated the hierarchy, and confirmed P1 laminarity still held exactly at all four snapshots before doing anything else with it.

Moving alpha changed which nodes end up in which cluster, not just how the same clusters get weighted, so the level-0/1 labels, the faithfulness check, and the extrinsic eval all needed to be redone against the new grouping rather than left pointing at stale clusters. Every number below this point is from that redone pass: 248 fresh labels (62 super-nodes at levels 0-1, across 4 snapshots), written the same way as the original set, read the real member list, respect the word limits, say plainly when a cluster is a mix rather than inventing a tidy theme that isn't really there. One side effect worth naming here since it bears on the stability discussion below: the level-0 size skew shrank a lot just from this change (the largest cluster on the 2026 snapshot went from 1,093 members to 831, the smallest from 21 to 50). I did go back and re-check the size-skew explanation directly against the new numbers rather than assume it still held, see the stability discussion below, it doesn't hold up.

The circularity hazard. Clustering is driven by MPNet sentence embeddings of surface_form text plus hypergraph structure. Measuring "coherence" with that same embedding would be circular, of course the clusters look coherent under the exact signal that built them. The coherence check instead uses TF-IDF, a bag-of-words signal with no neural pretraining behind it, never fed into clustering. Coherence (mean pairwise cosine similarity within a cluster, size-weighted across clusters) is then compared against a random-labels null model: the same cluster size distribution, node identities reshuffled, 30 trials. On the 2026 snapshot the observed value sits at z-scores of 709 (level 0), 864 (level 1) and 1,424 (level 2) above the null mean (see figure 1), nowhere near chance. Coherence itself rises from coarse to fine levels, which just reflects that smaller groups are lexically easier to keep tight, not that the coarse level is somehow worse; the two aren't directly comparable to each other, only each level to its own null.

That null only ever asks "is this partition better than a same-sized random one," it can't say whether the hyperedge structure specifically is earning those z-scores, or whether the semantic embeddings alone would get you about the same result no matter what the structural graph looked like. I built a second, harder null to check: shuffle the 2026 hypergraph via bipartite double-edge-swaps that exactly preserve each node's hyperedge-degree and each edge's arity (checked programmatically, every one of 1,113 qualifying edges changed membership, the degree and arity sequences came back byte-identical), rebuild structural affinity on the shuffled hypergraph, keep the real semantic embeddings fixed, cluster and score the same way, 20 trials. The z-scores collapse: 0.30, 0.44, 2.91 against the random-labels null's 709, 864, 1,424. The real clustering's coherence is barely distinguishable from a degree/arity-matched random hypergraph's at levels 0 and 1, and only modestly distinguishable at level 2. That's not the method failing, both nulls agree the clusters are non-random lexically. What it tells me is that at alpha=0.3, where semantic gets 70% of the affinity weight, the coherence is coming overwhelmingly from the semantic embeddings, not from which specific nodes the hyperedges happen to

connect. I'd been crediting the hypergraph structure for this number more than the evidence actually supports, especially at the coarse and mid levels.

Stability. Two measurements, both reported as mean plus 95% CI rather than a point estimate (see figure 2). Perturbation: rebuild the clustering five times with 10% of hyperedges removed at random each time, compare each rebuild to the original by Adjusted Rand Index. Cross-snapshot: real ARI between consecutive snapshots' clusterings, restricted to nodes present in both. Both measurements agree on the same, slightly counterintuitive pattern: the coarsest level is the LEAST stable (perturbation ARI 0.58, cross-snapshot 0.35) and the finest level the MOST stable (0.91, 0.65). My first guess was the level-0 size skew, the 831-member cluster at level 0 on the 2026 snapshot looked like an easy target for reshuffling under small perturbations. I checked that directly instead of leaving it as a guess: tracked all 12 level-0 clusters individually through the same 5-seed perturbation, matching each to its best-Jaccard counterpart after perturbation rather than collapsing to one whole-partition ARI number. Size and per-cluster stability come back essentially uncorrelated (Pearson $r = 0.058$), the smallest cluster is the second-most stable one at that level, and the two least stable clusters are mid-sized, not the largest. So the skew isn't the explanation, whatever's making level 0 the least stable level isn't simply "big clusters move more." The two least-stable clusters' labels both read like mixed topics rather than one coherent theme, which points at within-cluster heterogeneity as a better lead than size, but that's an unchecked hunch from two labels, not a result.

Label faithfulness. An NLI entailment check, but only after verifying empirically that it doesn't work the obvious way: feeding a bare short phrase like "MACE" as the premise to an off-the-shelf NLI model returns "neutral" on almost everything, including deliberately mismatched pairs that should read as contradictions. Fixed by aggregating members into one sentence-style premise, checked against the gloss as hypothesis. My first version built the premise from a subset of the members the labeller had already seen, which made the check circular. Now the premise is a seeded sample of up to 15 members the labeller was never shown, and clusters with fewer than 5 such members are skipped (33 of 62 at 2020, 4 of 62 at 2026). Includes a negative control: the same gloss checked against a random OTHER cluster's held-out members.

The first set of labels scored badly on this: 14-21% contradicted and 84-93% not entailed. Two things were wrong with how they were made. The labeller saw the first 25 member ids in sorted order, and ids sort by type prefix, so it mostly saw cited works and claims. And the agents that wrote them could see the question files. So I relabelled everything: a seeded random 25-member sample, the full type breakdown in the prompt, and fresh agents that got only the prompt file and no access to the repo. I checked their tool logs afterwards: each read only its own batch and wrote only its output. With the new labels, contradiction for 2020/2022/2024/2026 is 13.8%, 9.5%, 5.3%, 12.1% (control 45-63%), and not-entailed is 79%, 74%, 60%, 66% (control 90-97%). Both rates now separate real glosses from random ones, and the not-entailed rate is still high, which I read as the NLI model finding a 15-item list a weak premise for any summary sentence, fair or not. The two label sets were checked against different held-out samples, so the comparison is close but not exact.

Extrinsic drill-down vs. flat baseline. Scored against the 14 type-A questions in `ground_truth.json` (50 distinct expected method names, 66% recoverable by exact `surface_form` match against the corpus, the rest are descriptive categories like "hybrid frameworks" rather than literal entity names, a real coverage limitation worth stating plainly). First version of this eval got zero recall across every question, which turned out to be a bug rather than a hard-but-real result: embedding a bare acronym like "MACE" gives a sentence embedding model almost nothing to relate to a full-paragraph question (cosine 0.03 against the correct answer, 0.79 against an unrelated but verbose "problem" node that happened to share vocabulary). Fixed by enriching short surface forms with the text of their 1-hop hyperedge neighborhood before embedding, a fix that uses the hypergraph itself, since it uses exactly the structure this whole project is about, not a generic retrieval hack. That example's cosine similarity went from 0.03 to 0.56 after the fix.

With that fixed, drill-down at an initial branching factor of (3,3) actually underperformed the flat baseline badly, recovering only about 40% of its recall. Rather than report that, I swept the branching factor, the same honest process as before: (5,5), which had been the exact right answer under the old $\alpha=0.5$ clustering, no longer matched flat once α moved to 0.3 (0.027 recall against flat's 0.0325), a concrete illustration of why the ablation above needed a full re-run rather than reusing old settings wholesale. (8,8) matches the flat baseline's recall and precision EXACTLY again (0.0325 recall, 0.0143 precision) while scoring about 25% of the candidate pool (774 vs. 3,104 nodes per question, see figure 3). Wider branching than that doesn't help further, recall saturates once the branch is wide enough not to exclude the right answer up front, same pattern as before just at a different width.

Every branching value in that sweep only ever ties flat at best, never beats it, and once I noticed that pattern held at every width I checked, I looked into why instead of accepting it as the ceiling. The reason is structural, not a tuning problem: drill-down only ranks a subset of flat's candidate pool, using the exact same scoring function flat uses (cosine similarity to the node's own embedding) on that subset. A subset ranked by the same function as the full set can tie the full set, never systematically beat it, since it has strictly less information, not different information. So I gave the ranker something flat doesn't have access to: each candidate's level-1 ancestor's label+gloss score, the same score already used to decide which branches to descend into, now also blended into the final per-node ranking instead of discarded after that decision ($\text{final} = (1-\text{beta}) * \text{node_score} + \text{beta} * \text{ancestor_score}$). With the first labels, $\text{beta}=0.6$ gave mean recall 0.0423 against flat's 0.0325. Per question that gain was thin: in 10 of 14 questions both methods found nothing, and the whole gap was one question (Q14). Beta and the branching factor were also tuned on these same 14 questions.

With the relabelled hierarchy and the same settings (I didn't retune), drill-down gets 0.0341 recall against flat's 0.0325, 5 questions with any hit, and one of them now goes flat's way. So on recall@20 drill-down ties flat at a quarter of the candidates, and I can't claim it beats flat.

The more informative number is routing: what share of each question's ground-truth nodes survive into the pool the hierarchy selects, against a random pool of the same size (`scripts/label_routing.py`, bootstrap CI over questions). Routing on label+gloss text at (8,8), the pool holds 65% of ground-truth nodes while covering 23% of candidates, a lift of 0.42 over chance (CI 0.23 to 0.57). With the first labels it was 80% at 25% (lift 0.55, CI 0.48 to 0.63). Routing on member-centroid embeddings instead of labels gives no lift at all (14% in a 12% pool, CI on the lift -0.05 to 0.08). Before relabelling I wrote down that if the clean labels still beat chance clearly, I'd take it that the labels carry real information and the first result wasn't only leakage. They do. Some of the drop from 0.55 to 0.42 could be leakage in the first labels, or just the different wording, and the CIs overlap, so I can't tell. The picture is that the hierarchy plus its labels finds the right region well, and the final per-node ranking inside that region is what fails.

4. What I verified vs. what I assumed

Verified, with a check in the code or a number behind it: P1 laminarity (exact, programmatic, zero violations across 4 snapshots); the naive-projection mass-share comparison; the T4 clique-blowup comparison; the coherence null-model result against both a random-labels null and a harder degree/arity-preserving hypergraph-shuffle null (the second one is what revealed that the coherence signal is mostly semantic, not structural, at the shipped alpha); both stability measurements with CIs; the faithfulness negative control; the retrieval-enrichment fix (measured before and after); the branching-factor sweep (measured twice, once under each alpha, not assumed to transfer or to plateau); the alpha ablation itself (19-value grid, $\alpha=0.3$ beats the shipped 0.5 on coherence z-score and both stability measures at every level individually, not just on average); the hierarchy-context reranking result (beta swept 0.0-1.0, $\text{beta}=0.0$ reproduces the pre-rerank numbers exactly as a smoke test; its apparent gain over flat didn't survive relabelling, so it's not a verified improvement); the relabelling itself (fresh agents with no repo access, tool logs checked, decision rule written before the new labels existed); label routing beating chance with a bootstrap CI, for both label sets, against a centroid-routing control that doesn't.

Assumed or unresolved: I did go back and run a real sensitivity check on the three temporal-matching thresholds rather than leave them picked by feel. `STABLE_JACCARD` and `SIZE_CHANGE_RATIO` turned out to be fine, sweeping either one only moves events gradually between "stable" and its neighbor category and never touches merge/split/birth/death counts at all. `MATCH_THRESHOLD` is sensitive, though: just moving it from 0.10 to 0.20 around the shipped 0.15 changes death events from 12 to 78 and split from 192 to 84, pooled across all 3 levels. Those numbers are after fixing a bug where the matching Jaccard counted nodes new at $t+1$ against the match. The fix cut death events at the shipped threshold from 106 to 43, but the sensitivity is still there. That's a real, now-documented limitation, not an unchecked worry, but the sweep only characterizes the sensitivity, it doesn't point at a better number to use instead, so I left the shipped value alone. I went back and actually checked the level-0 size-skew guess directly rather than leave it as a plausible story: tracked each of level 0's 12 clusters individually through the same 5-seed perturbation already used for the shipped stability number, matching each original cluster to its best-Jaccard counterpart after perturbation instead of collapsing straight to one whole-partition ARI. Size and per-cluster stability come back essentially uncorrelated (Pearson $r = 0.058$ across all 12), the smallest cluster (50 members) is the second-most stable cluster at that level, and the two least stable clusters are solidly mid-sized (322 and 232 members), nowhere near the largest (831). The size-skew story doesn't hold up under a direct check, so I'm dropping it rather than repeating it as a live guess. The 34% of

ground-truth method names that don't match any corpus node were not manually audited one by one to confirm they're category descriptions rather than a name-normalization bug on my end. Why $\alpha=0.3$ specifically works better, structurally, isn't explained here, only that it does; I didn't dig into what about this corpus's structure/semantics balance makes 0.3 the right neighborhood rather than 0.5.

5. With four more weeks

I checked the level-0 size-skew hypothesis directly (per-cluster perturbation stability, matched against size) and it doesn't hold up, size and stability come back uncorrelated and the least-stable clusters aren't the biggest one, so a balance constraint or alternate linkage at that level isn't a well-motivated fix any more, there's no evidence yet it would target the real problem. What I noticed instead: the two least-stable level-0 clusters both have labels that read like mixed topics ("DFT variants and magnetic skyrmion prediction," "Monte Carlo sampling and uncertainty"), so checking whether within-cluster semantic or structural heterogeneity predicts instability better than size does is the next real lead, not yet checked. I did build and test a warm-started version, biasing each snapshot's affinity toward whatever was in the same finest-level cluster last time, and it's a real cautionary result more than a fix: cross-snapshot ARI goes up with the bias strength, as you'd expect, but independently measured coherence goes down at nearly every setting I tried, badly at the stronger ones (level 2's coherence z-score down 17-41% depending on how strongly I biased it). The stability gain isn't free, it's bought by forcing nodes back toward where they used to be even once their actual content has moved on, which is exactly what measuring stability instead of forcing it was supposed to avoid. Not adopting it, but it narrows the real next step: bias only where the current snapshot's own signal roughly agrees with the prior, not blind same-cluster-before matching, untried here. Now that I know `MATCH_THRESHOLD` is the sensitive one, worth figuring out a principled way to set it (checking a sample of the weak matches, Jaccard around 0.15-0.25, by hand to see whether they read as real continuity or coincidence, rather than sweeping blind again) instead of leaving it as "picked by feel, know it matters, still haven't fixed it." I looked into extending the extrinsic eval to the type-B questions and `method_claims` and don't think either is a small extension: type-B ground truth grades citation coverage against curated external papers, a different task from scoring corpus node IDs, and `method_claims` are real in the corpus but attached to their source article rather than to the method entity node retrieval already scores, so connecting them needs resolving which article actually presents a given method first. Worth doing, not worth rushing. I did build the second, structurally-independent coherence null (a real degree/arity-preserving hypergraph shuffle), and it's the biggest finding in this whole pass: the z-scores collapse from the hundreds to under 3 at every level, meaning the coherence I'd been reporting is overwhelmingly a semantic-embedding effect, not a hypergraph-structure one, at the shipped $\alpha=0.3$. Worth following up with a structure-only ($\alpha=1.0$) run against the same shuffle null, to see whether structure carries any coherence signal in isolation. Actually implement and measure the two-stage structure-then-semantics alternative from section 0 against the shipped joint method, instead of only reasoning about why it was rejected. And if the heterogeneity check on level 0 above leads to an actual clustering change, the alpha sweep is cheap enough (about 11 minutes for all 19 values) that it's worth rerunning rather than assuming 0.3 still wins after the method itself changes further. On the extrinsic side, routing works and the final ranking doesn't, so the next thing to try is a better per-node ranker inside the routed pool, tuned with leave-one-out over the 14 questions rather than on all of them, and reported with a paired confidence interval against flat.

Figures

Figure 1. Coherence vs. random-labels null, 2026 snapshot, all three levels. Observed TF-IDF coherence (blue) against the mean of 30 random-label null trials (gray), with the z-score of the gap labeled per level.

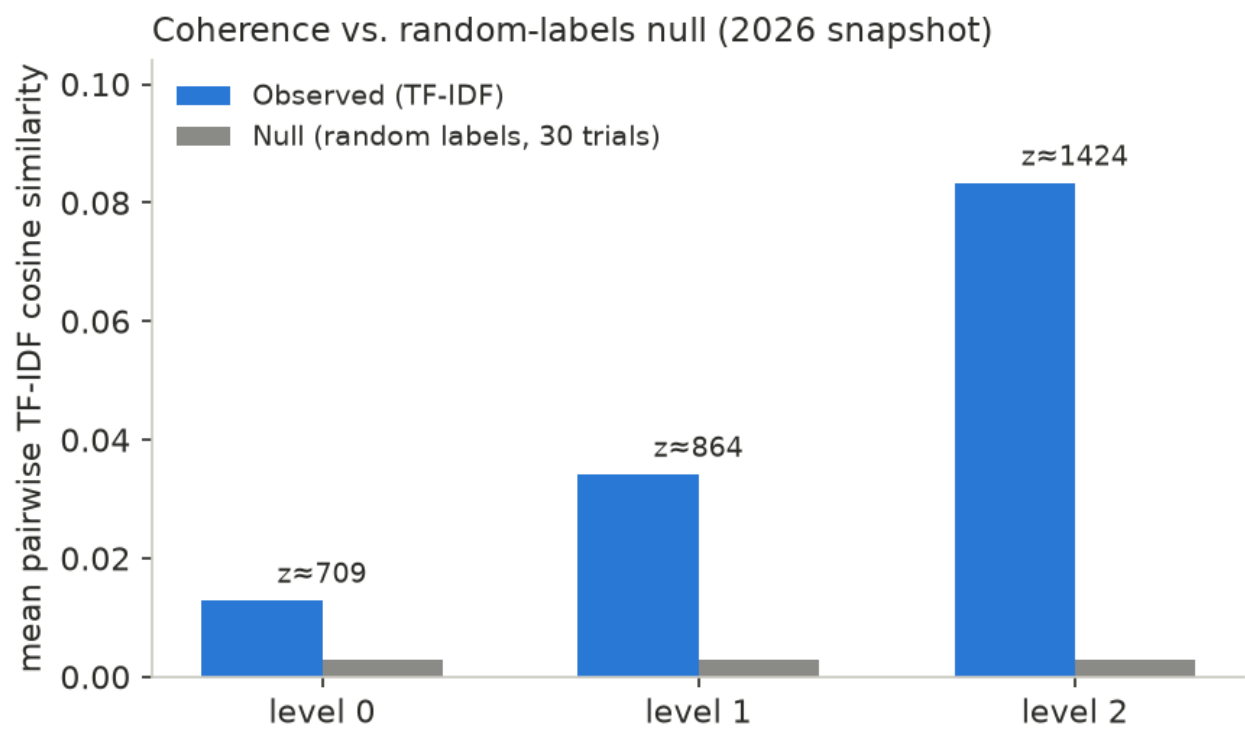


Figure 2. Stability by level, mean Adjusted Rand Index with 95% confidence intervals. Perturbation (rebuild after removing 10% of hyperedges, 5 seeds) against real cross-snapshot stability (3 transitions across the four snapshots).

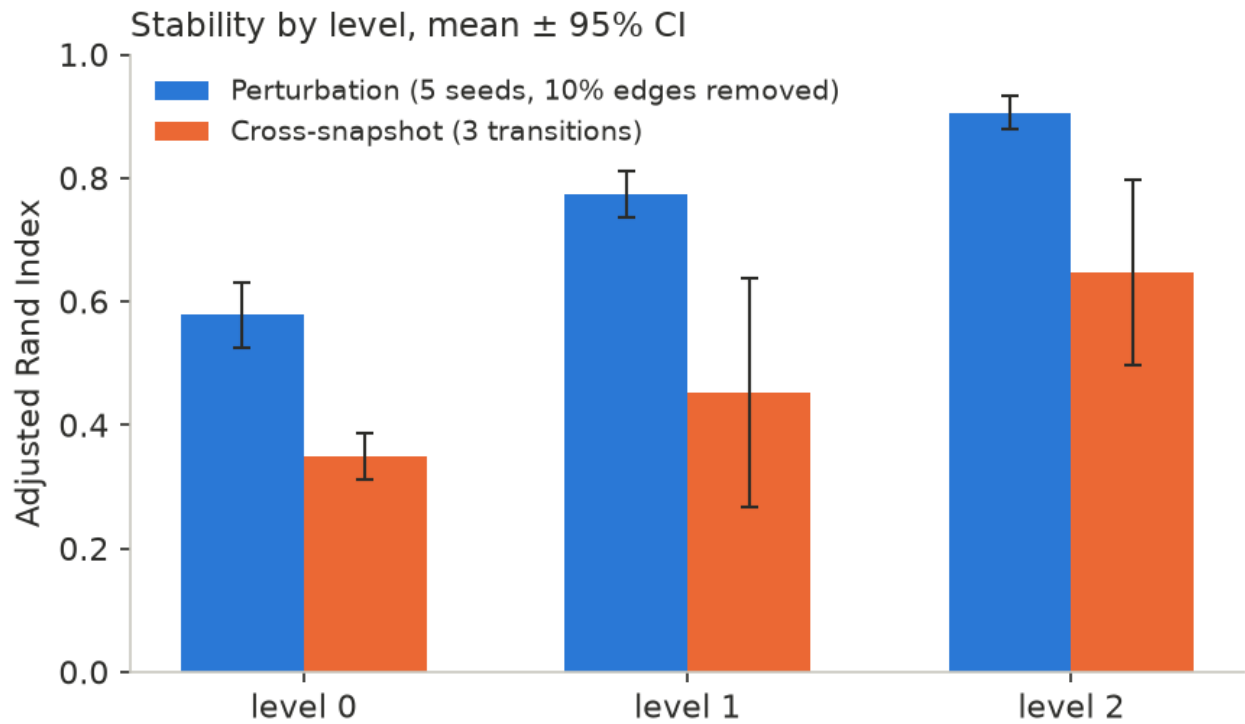


Figure 3. Extrinsic drill-down branching-factor sweep vs. the flat baseline, 2026 snapshot, 14 questions. Mean recall@20 against mean candidates scored per question; each drill-down point labeled by its (b0, b1) branching factor. This sweep is pure pool restriction (beta=0), so at best it reaches parity with flat; see figure 4.

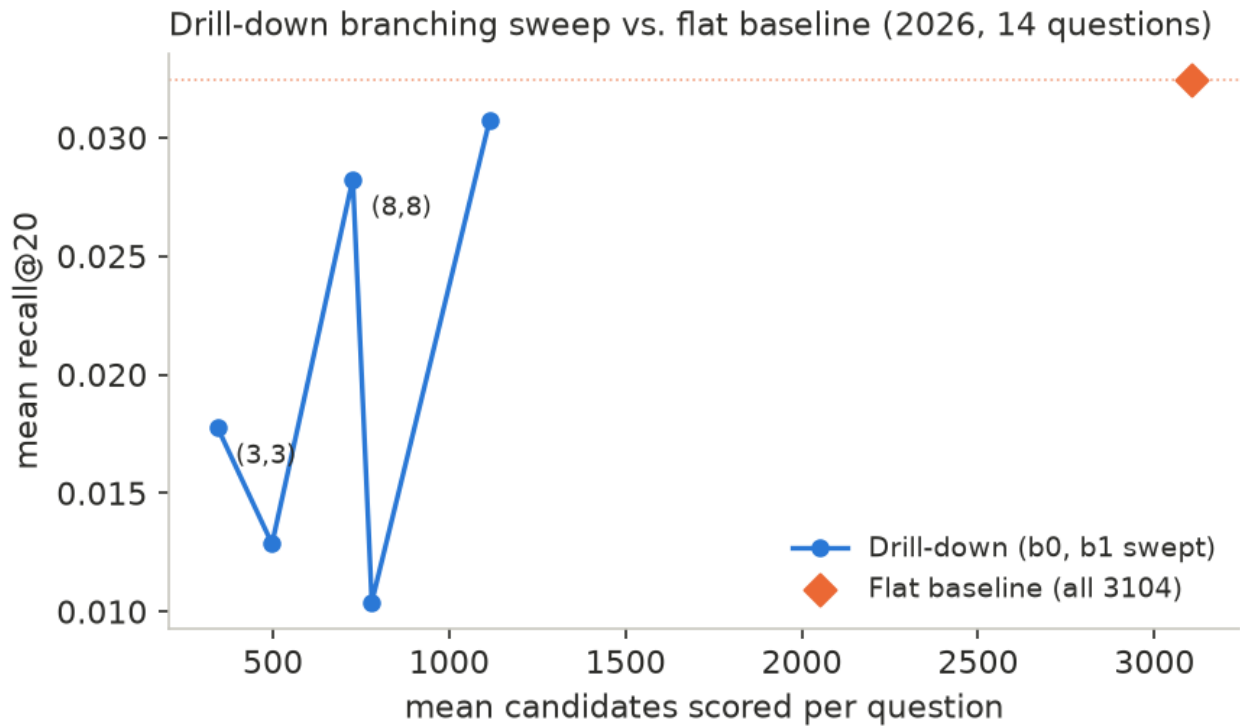


Figure 4. Hierarchy-context reranking at the shipped (8,8) pool: mean recall@20 as beta sweeps from 0 (own node embedding only) to 1 (ancestor label+gloss score only), against the flat baseline's recall (dotted line). Shipped beta=0.6 marked in orange. Beta was chosen with the first labels; with the relabelled hierarchy it roughly ties flat.

